

Programmation système TdTp 1 : Exercice de Synthèse n°1

1. Exemple

Dans un processus, un fichier ouvert possède une tête de lecture/écriture. Par défaut, celle-ci est placée en tête du fichier à l'ouverture et est automatiquement déplacée après chaque lecture ou écriture.

ecriture.c

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <sys/fcntl.h>
#include <unistd.h>

main(int argc, char **argv)
{
    int desc, nb_lu;
    char tab[8];

    if (argc != 2) {
        printf("Usage : %s file\n", argv[0]);
        exit(1);
    }

    if ((desc = open(argv[1], O_RDONLY)) == -1) {
        perror("Ouverture impossible :");
        exit(2);
    }

    while((nb_lu = read(desc, tab, 7*sizeof(char))) > 0) {
        tab[nb_lu]='\0';
        printf("nb_lu : %d %s\n", nb_lu, tab);
    }
    close(desc);
}
```

lecture.c

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <sys/fcntl.h>
#include <unistd.h>           // pour exit

main(int argc, char **argv)
{
    int desc;
    char c;

    if (argc != 2) {
        printf("Usage : %s file\n", argv[0]);
        exit(1);
    }

    if ((desc = open(argv[1], O_CREAT|O_WRONLY|O_TRUNC, 0600)) == -1) {
        perror("Ouverture impossible :");
        exit(2);
    }

    while(scanf("%c", &c) != EOF) { // EOF renvoyé si <ctrl>D correspond à -1
        write(desc, &c, 1);
    }
    close(desc);
}
```

2. Ecrire un programme contenant une fonction « *afficher (..)* » permettant de copier le contenu d'un fichier, fichier désigné par son nom externe dans le système, dans un autre fichier associé à la sortie standard de l'application (entrée n°1 de la table des fichiers ouverts). Un contrôle doit-être effectué sur l'existence du fichier source.

3. Utiliser la fonction définie en question 2 pour réaliser la commande "cat" du système UNIX. Cette commande permet l'affichage, à l'écran de l'utilisateur d'un ou de plusieurs fichiers spécifiés en paramètres.

Exemple : Cat fichier1 fichier2

4. Modifier la fonction de la question3 en introduisant la possibilité, pour l'utilisateur, d'introduire une éventuelle redirection.

Exemple: Cat fich1 fich2 fich3>fich-dst

Programmation système

TdTp 2 : Exercice de Synthèse n°2

Thème : Manipulation des primitives sur les processus : Fork wait exit.

1- Exemples : exécuter les programmes suivants :

Premier exemple

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

main() {
    pid_t pid;

    pid = fork();
    switch(pid) {
        case -1 : perror("erreur :");break;
        case 0 : printf("processus fils : %d %d\n", getpid(), getppid());break;
        default :
            printf("processus père : %d\n", getpid());
    }
    printf("fin : %d \n", getpid());
}
```

Deuxième exemple

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

main() {
    pid_t pid;
    int i;

    printf("Bonjour à tous\n");
    pid = fork();
    printf("je suis le processus : %d, pid : %d\n", getpid(), pid);
    sleep(1);

    for(i=0; i<3; i++) {
        printf("proc : %d i : %d\n", getpid(), i);
        sleep(1);
    }

    if (pid == 0) {
        printf("je suis le fils et je termine\n");
    } else {
        printf("je suis le père et je termine\n");
    }
}
```

2- Reprendre le problème de la question 2 du 1er TD, en se limitant à un seul fichier à afficher et en confiant ce travail à un processus fils.

3- Généraliser la question 2 à plusieurs fichiers (sans redirection), chaque fichier à afficher étant traité par un processus fils spécifié. (chaque processus fils ne doit afficher qu'un seul fichier.).

Dans le cas où un fichier à afficher n'existe pas, le processus associé doit afficher un message d'erreur et le processus père doit alors stopper la commande.

Programmation système TdTp 3 : Exercice de Synthèse n°3

Thème: Manipulation des primitives sur les images miroir et sur les tubes de communication

1. Exemples : exécuter les programmes suivants :

```
void main()
{
    execl("/bin/ls", "ls", "-l", (char *) 0);
    /* avec execlp, le premier argument peut n'être
    que ls et non /bin/ls */
    printf("pas d'impression\n");
}
```

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

main () {
    pid_t pid;
    int status;

    pid=fork();
    if (pid == 0) {
        execl("/bin/ls", "ls", "-l", NULL);
    }
    wait(&status);
    printf("Fils terminé !\n");
}
```

2- Exercice

Ecrire la commande Unix "run" Définie de la manière suivante
run fichier.c

Sémantique

La commande "run" réalise d'abord la compilation du source C. dans le cas où la compilation se termine correctement, l'exécution du fichier exécutable est immédiatement lancée; les messages d'erreurs sont affichés sur la sortie standard (en utilisant la commande UNIX, pg);

Un éditeur de textes (exemple "vi") est ensuite appelé afin de modifier le texte source.

3- Exercice

Programmer la commande '**compter**' qui fournit le nombre de fichiers que possède un utilisateur dans une hiérarchie en manipulant les primitives des tubes de communication

Appel : compter hiérarchie utilisateur

Traitement réalisé : ls -Ral hierarchie | grep utilisateur | wc -l

Programmation système

TdTp 4 : Exercice de Synthèse n°4

Thème : Manipulation des primitives sur les Threads

1. Exemple

```
# define _REENTRANT

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

void afficher (int n , char lettre)
{
    int i,j;
    for (j=1; j<n; j++) {
        for (i=1; i < 10000000; i++){
            printf ("%c", lettre);
            fflush (stdout);
        }
    }
}

void *threadA (void *inutilise)
{
    afficher (100, 'A');
    printf("\n Fin du thread A\n");
    fflush (stdout);
    pthread_exit(NULL);
}

void *threadC (void *inutilise)
{
    afficher (150, 'C');
    printf("\n Fin du thread C\n");
    fflush (stdout);
    pthread_exit(NULL);
}

void *threadB (void *inutilise)
{
    pthread_t thC;
    pthread_create(&thC, NULL, threadC, NULL);

    afficher (100, 'B');

    printf("\n le thread B attend la fin du thread C  \n");
    pthread_join (thC, NULL);
    printf("\n Fin du thread B\n");
    fflush (stdout);
    pthread_exit(NULL);
}

int main() {
    {
        int i;
        pthread_t thA, thB;

        printf ( " Creation du thread A" ) ;
        pthread_create (&thA , NULL , threadA , NULL) ;
        pthread_create (&thB , NULL , threadB , NULL) ;
        sleep(1) ;

        // attendre que les threads aient termine

        printf (" Le thread principal attend que les autres se terminent\n " ) ;
        pthread_join(thA ,NULL) ;
        pthread_join(thB ,NULL) ;

        exit(0);
    }
}
```

2. Exercice

- a. Ecrire un programme ayant le comportement suivant :

Des threads sont créés (leur nombre étant passé en paramètre lors du lancement du programme) ;

Chaque thread affiche un message (par exemple « hello world ! ») ;

Le thread « principal » attend la terminaison des différents threads créés.

- b. Modifiez le programme de la question précédente pour que chaque thread affiche :
son PID (avec `getpid()`) ;
La valeur opaque retournée par `pthread_self`, par exemple avec :
`printf("%p \n", (void *) pthread_self ()) ;`
- c. Modifier le programme de la question précédente pour passer son numéro d'ordre à chaque thread. Chaque thread doit ensuite l'afficher. Vérifier que le numéro d'ordre affiché par chaque thread est bien différent (corriger votre programme le cas échéant).

Programmation système TdTp 5 : Exercice de Synthèse n°5

Thème : Manipulation des primitives sur les Threads en Posix

Exemple : Le programme sem-posix.c montre l'utilisation de sémaphores Posix. Expliquer ?

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <unistd.h>
#include <semaphore.h>

sem_t my_sem;
#define SEMINITVALUE 1
void * critical_section ( void * ) ;
#define MAXTHREADS 100

int main (int argc , char *argv [ ])
{
    int n, i, error;
    pthread_t threadid [MAXTHREADS] ;
    if (( argc != 2 ) || ( ( n = atoi (argv [1])) <= 0 ))
    {
        printf("Usage : %s nombre de threads\n" , argv[0]);
        exit(1) ;
    }

    // initialiser le sémaphore
    sem_init(&my_sem, 0 , SEMINITVALUE) ;
    // création des threads
    for (i = 0; i < n; ++ i)
        pthread_create (&threadid [i], NULL, critical_section, (void *)i );
    // attendre la fin des threads
    for (i = 0; i < n; ++ i)
        pthread_join (threadid [i], NULL);
    exit(0);
}

void * critical_section (void *arg )
{
    int myindex, i, n;
    // extraire l'index des threads
    myindex = (int) arg;
    sem_wait(&my_sem) ;
    // section critique
    for (i = 1; i < 5; i ++ )
    {
        printf ("%d : %d\n" , myindex , i);
        if ( ( n=sleep (1)) != 0 ) // dormir 1 seconde
            printf ( "interrupted , no of secs left %d\n" , n );
    }
    // quitter section critique
    sem_post (&my_sem);
    pthread_exit (NULL);
}
```

sekkaki> gcc -o posix-sem posix-sem.c -lpthread

sekkaki> posix-sem 2

0:1

0:2

0:3

0:4

1:1

1:2

1:3

1:4

sekkaki>

Exercice

Producteur/consommateur avec sémaphores. La solution du problème au moyen des sémaphores Posix utilise trois sémaphores dans le programme `prod-cons.c`. Le premier sémaphore, nommé `plein`, compte le nombre d'emplacements occupés. Il est initialisé à 0. Le second, nommé `vide`, compte le nombre d'emplacements libres. Il est initialisé à N (la taille du tampon). Le dernier, nommé `mutex`, assure l'exclusion mutuelle pour l'accès au tampon (lorsqu'un processus utilise le tampon, les autres ne peuvent pas y accéder).